

JavaScript Developer

Interview Preparation Guide · Celestial Systems Pvt. Ltd.

Date: 18 March 2026	Time: 9:15 AM onwards	Stipend: ■20,000/month (3-month training)	CTC: ■4–5 LPA (post-training)
----------------------------	------------------------------	--	--------------------------------------

Round 1 Objective + Coding (Online)	Round 2 Technical Interview	Round 3 Final Tech Interview with Manager
---	---------------------------------------	---

Section 1 · Aptitude / Objective Questions (MCQs)

15–20 Multiple Choice Questions | JavaScript Fundamentals

1. What will `typeof null` return in JavaScript?

- A) null
- B) undefined
- C) object
- D) string

✓ **Correct Answer: C**

Hint: This is a well-known JavaScript quirk — `null`'s type is 'object' due to a legacy bug.

2. Which of the following correctly declares a block-scoped variable?

- A) `var x = 10;`
- B) `let x = 10;`
- C) `global x = 10;`
- D) `static x = 10;`

✓ **Correct Answer: B**

Hint: `let` and `const` are block-scoped; `var` is function-scoped.

3. What is the output of: `console.log(0.1 + 0.2 === 0.3)`?

- A) true
- B) false
- C) NaN
- D) undefined

✓ **Correct Answer: B**

Hint: Floating-point precision issue — $0.1 + 0.2 = 0.30000000000000004$.

4. Which method removes the last element from an array and returns it?

- A) `shift()`
- B) `unshift()`
- C) `pop()`
- D) `splice()`

✓ **Correct Answer: C**

Hint: `pop()` → removes last; `shift()` → removes first.

5. What does the `===` operator check?

- A) Value only
- B) Reference only
- C) Value and type
- D) Prototype chain

✓ **Correct Answer: C**

Hint: Strict equality checks both value AND type without coercion.

6. What is the result of `[] + []` in JavaScript?

- A) []
- B) 0
- C) ""
- D) null

✓ **Correct Answer: C**

Hint: Both arrays are coerced to empty strings, so `"" + "" = ""`.

7. Which of these is NOT a primitive data type in JavaScript?

- A) string
- B) boolean
- C) object
- D) symbol

✓ **Correct Answer: C**

Hint: Primitive types: string, number, bigint, boolean, undefined, symbol, null. Object is not primitive.

8. What does `Array.prototype.map()` return?

- A) undefined
- B) The original array
- C) A new array
- D) A boolean

✓ **Correct Answer: C**

Hint: `map()` always returns a new array; it doesn't mutate the original.

9. Which keyword is used to handle errors in `async/await`?

- A) catch
- B) try/catch
- C) error
- D) finally

✓ **Correct Answer: B**

Hint: Use `try { await ... } catch(e) { ... }` for async error handling.

10. What is the output of `typeof undefined`?

- A) null
- B) object
- C) undefined
- D) string

✓ **Correct Answer: C**

Hint: `typeof undefined` returns the string "undefined".

11. Which HTTP method is idempotent and used to update a resource completely?

- A) POST
- B) PATCH
- C) PUT
- D) DELETE

✓ **Correct Answer: C**

Hint: PUT replaces the entire resource; PATCH does partial updates.

12. In JavaScript, which of the following creates a Promise?

- A) `new Promise((resolve, reject) => {})`
- B) `Promise.new()`
- C) `create Promise()`
- D) `async()`

✓ **Correct Answer: A**

Hint: Promises are created with `new Promise((resolve, reject) => {...})`.

13. What will `console.log(1 + '2')` output?

- A) 3
- B) 12
- C) '12'
- D) NaN

✓ **Correct Answer: C**

Hint: When a number is added to a string, JavaScript coerces the number to a string.

14. Which array method is used to combine two arrays?

- A) join()
- B) concat()
- C) merge()
- D) push()

✓ **Correct Answer: B**

Hint: concat() returns a new array; the spread operator [...a, ...b] also works.

15. What is event bubbling in JavaScript?

- A) Event fires only on the target element
- B) Event propagates from child to parent
- C) Event propagates from parent to child
- D) Event is cancelled automatically

✓ **Correct Answer: B**

Hint: Bubbling = child → parent. Capturing = parent → child.

16. Which of the following is used to prevent default browser behavior?

- A) event.stop()
- B) event.cancel()
- C) event.preventDefault()
- D) event.disable()

✓ **Correct Answer: C**

Hint: event.preventDefault() stops the browser's default action (e.g., form submit).

17. What does `JSON.stringify()` do?

- A) Parses a JSON string into an object
- B) Converts a JavaScript object to a JSON string
- C) Validates JSON format
- D) Encrypts JSON data

✓ **Correct Answer: B**

Hint: JSON.stringify(obj) → string; JSON.parse(str) → object.

18. Which of the following is a JavaScript framework/library?

- A) Django
- B) Laravel
- C) React
- D) Flask

✓ **Correct Answer: C**

Hint: React is a JavaScript library for building UIs. Django/Flask are Python; Laravel is PHP.

19. What is the purpose of `localStorage` in a browser?

- A) Store data on the server
- B) Store data permanently in the browser
- C) Store session-only data in the browser
- D) Store cookies

✓ **Correct Answer: B**

Hint: localStorage persists even after browser close; sessionStorage is cleared on tab close.

20. Which CSS property is used to make an element invisible but still occupy space?

- A) display: none
- B) opacity: 0
- C) visibility: hidden
- D) Both B and C

✓ **Correct Answer: D**

Hint: Both opacity:0 and visibility:hidden hide the element but keep its space; display:none removes it.

Section 2 · Technical Concept Questions

20 Theory-Based Questions | Deep JavaScript & Web Knowledge

1. What is the difference between `var`, `let`, and `const`?

Answer: `var` is function-scoped and hoisted; `let` is block-scoped and not hoisted (TDZ); `const` is block-scoped and cannot be reassigned. All three are hoisted but `let/const` stay in the Temporal Dead Zone until declared.

2. Explain closures in JavaScript with an example.

Answer: A closure is a function that retains access to its outer scope even after the outer function has returned.
Example: a counter function where the inner `increment()` accesses the outer `count` variable.

3. What is the Event Loop and how does it work?

Answer: The Event Loop continuously checks the Call Stack and Message Queue. It moves callbacks from the queue to the stack only when the stack is empty. This enables asynchronous non-blocking behavior in JavaScript's single-threaded model.

4. What is hoisting in JavaScript?

Answer: Hoisting is JavaScript's behavior of moving declarations (not initializations) to the top of the scope. `var` declarations are hoisted and initialized to `undefined`; function declarations are fully hoisted; `let/const` are hoisted but not initialized (TDZ).

5. What is the difference between `==` and `===`?

Answer: `==` uses type coercion (loose equality); `===` compares both value and type without coercion (strict equality). Best practice is to always use `===`.

6. Explain Promises and how they differ from callbacks.

Answer: A Promise represents a future value. It avoids callback hell by chaining `.then()/catch()`. Promises provide better error handling and readability. `async/await` is syntactic sugar over Promises.

7. What is the prototype chain in JavaScript?

Answer: Every JS object has a `__proto__` link to its prototype object. When a property is not found on the object, JS looks up the chain until null is reached. This is the basis of prototype-based inheritance.

8. What are arrow functions and how do they differ from regular functions?

Answer: Arrow functions have no own `this`, `arguments`, or `prototype`. They inherit `this` from the enclosing lexical scope. They cannot be used as constructors and are ideal for callbacks.

9. Explain `async/await` and error handling with it.

Answer: `async` functions always return a Promise. `await` pauses execution until the Promise resolves. Errors are caught using `try/catch` blocks, which provide synchronous-style error handling for `async` code.

10. What is the difference between `null` and `undefined`?

Answer: `undefined` means a variable is declared but not assigned. `null` is an intentional absence of value, assigned explicitly. `typeof null === 'object'` (bug), `typeof undefined === 'undefined'`.

11. What is DOM manipulation? Give examples of common methods.

Answer: DOM manipulation is changing HTML/CSS via JavaScript. Common methods: `getElementById()`, `querySelector()`, `createElement()`, `appendChild()`, `innerHTML`, `addEventListener()`, `classList.add/remove/toggle()`.

12. What are Higher-Order Functions? Give examples.

Answer: Functions that take other functions as arguments or return functions. Examples: `map()`, `filter()`, `reduce()`, `forEach()`. These enable functional programming patterns in JS.

13. What is the difference between `call()`, `apply()`, and `bind()`?

Answer: All three set `this` explicitly. `call(ctx, a, b)` — invokes immediately with individual args; `apply(ctx, [a,b])` — invokes with array of args; `bind(ctx)` — returns a new function with `this` bound, doesn't invoke immediately.

14. What is event delegation?

Answer: Attaching a single event listener to a parent element instead of multiple children. It uses event bubbling — the parent catches events from children via `event.target`. Efficient for dynamic lists.

15. Explain the concept of 'this' in JavaScript.

Answer: `this` refers to the object that is executing the function. Its value depends on how the function is called: method call (the object), standalone call (global/undefined in strict), arrow function (lexical scope), new (the new instance).

16. What is the difference between synchronous and asynchronous JavaScript?

Answer: Synchronous code executes line by line, blocking further execution. Asynchronous code (callbacks, Promises, async/await) allows other operations to run while waiting for a task (I/O, API calls).

17. What are REST API principles?

Answer: REST is stateless, client-server, uniform interface, cacheable. Uses HTTP methods: GET (read), POST (create), PUT/PATCH (update), DELETE. Resources are identified by URLs. Responses are typically JSON.

18. What is CORS and why does it occur?

Answer: Cross-Origin Resource Sharing is a security mechanism. Browsers block requests to a different origin (domain/port/protocol) unless the server sends appropriate Access-Control-Allow-Origin headers.

19. What is the difference between `localStorage`, `sessionStorage`, and cookies?

Answer: localStorage: persists until manually cleared, ~5MB. sessionStorage: cleared on tab close. Cookies: sent with every HTTP request, small size (~4KB), can be server-set, support expiry and HttpOnly flags.

20. What is a JavaScript module and how do you use ES6 imports/exports?

Answer: Modules allow code splitting and reuse. Use `export` to expose, `import` to consume. Named exports: export const fn = ...; Default export: export default fn; Import: import {fn} from './file' or import fn from './file'.

Section 3 · Coding Questions

10 Coding Problems with Difficulty Levels

1. Reverse a String

Easy

Write a function that takes a string and returns it reversed.

Input: 'hello' → Output: 'olleh'

```
function reverseString(str) {  
  return str.split("").reverse().join("");  
}  
  
console.log(reverseString("hello")); // "olleh"
```

2. Check Palindrome

Easy

Write a function to check if a given string is a palindrome.

Input: 'racecar' → Output: true

```
function isPalindrome(str) {  
  return str === str.split("").reverse().join("");  
}  
  
console.log(isPalindrome("racecar")); // true
```

3. Find Largest Number in Array

Easy

Given an array of numbers, return the largest one.

Input: [3, 1, 9, 4] → Output: 9

```
function findLargest(arr) {  
  return Math.max(...arr);  
}  
  
console.log(findLargest([3, 1, 9, 4])); // 9
```

4. Count Vowels in a String

Easy

Write a function that counts the number of vowels in a string.

Input: 'hello world' → Output: 3

```
function countVowels(str) {  
  return (str.match(/[aeiou]/gi) || []).length;  
}  
  
console.log(countVowels("hello world")); // 3
```

5. Flatten a Nested Array

Medium

Write a function to flatten a deeply nested array.

Input: [1, [2, [3, [4]]]] → Output: [1, 2, 3, 4]

```
function flatten(arr) {  
  return arr.flat(Infinity);  
  // Or recursively:  
  // return arr.reduce((acc, val) =>  
  //   Array.isArray(val) ? acc.concat(flatten(val)) : acc.concat(val), []);  
}  
console.log(flatten([1, [2, [3, [4]]]])); // [1, 2, 3, 4]
```

6. Debounce Function

Medium

Implement a debounce function that delays the execution of a given function by a specified wait time. This is a very commonly asked question in frontend interviews.

```
function debounce(fn, wait) {  
  let timer;  
  return function(...args) {  
    clearTimeout(timer);  
    timer = setTimeout(() => fn.apply(this, args), wait);  
  };  
}  
// Usage:  
const debouncedSearch = debounce(search, 300);
```

7. Group Anagrams

Medium

Given an array of strings, group all anagrams together.

Input: ['eat','tea','tan','ate','nat','bat'] → [['eat','tea','ate'],['tan','nat'],['bat']]

```
function groupAnagrams(strs) {  
  const map = {};  
  for (let s of strs) {  
    const key = s.split('').sort().join('');  
    if (!map[key]) map[key] = [];  
    map[key].push(s);  
  }  
  return Object.values(map);  
}
```

8. Fetch Data and Handle Errors (async/await)

Medium

Write an async function that fetches user data from a REST API, handles errors gracefully, and logs the result.

```
async function fetchUser(id) {  
  try {  
    const res = await fetch(`https://api.example.com/users/${id}`);  
    if (!res.ok) throw new Error(`HTTP error: ${res.status}`);  
    const data = await res.json();  
    console.log(data);  
    return data;  
  } catch (err) {  
    console.error('Fetch failed:', err.message);  
  }  
}
```

9. Implement a Curry Function

Hard

Write a curry function that converts a function of N arguments into a chain of N functions each taking one argument.

add(1)(2)(3) should return 6.

```
function curry(fn) {  
  return function curried(...args) {  
    if (args.length >= fn.length) {  
      return fn.apply(this, args);  
    }  
    return function(...args2) {  
      return curried.apply(this, args.concat(args2));  
    };  
  };  
}  
  
const add = curry((a, b, c) => a + b + c);  
console.log(add(1)(2)(3)); // 6
```

10. Implement a Custom Promise.all()

Hard

Implement your own version of `Promise.all()` that resolves when all promises resolve, or rejects as soon as one rejects.

```
function myPromiseAll(promises) {  
  return new Promise((resolve, reject) => {  
    const results = [];  
    let completed = 0;  
    if (!promises.length) return resolve([]);  
    promises.forEach((p, i) => {  
      Promise.resolve(p)  
        .then(val => {  
          results[i] = val;  
          completed++;  
          if (completed === promises.length) resolve(results);  
        })  
        .catch(reject);  
    });  
  });  
}
```

Section 4 · Practical / Scenario-Based Questions

10 Real-World Problem-Solving Questions

1. A webpage is loading very slowly. How would you debug and optimize its performance?

Approach: Use Chrome DevTools (Network, Performance tabs) to identify bottlenecks. Solutions: lazy loading images, code splitting, minifying CSS/JS, caching, reducing DOM size, using a CDN, deferring non-critical JS, eliminating render-blocking resources.

2. You need to build a search bar that queries an API as the user types. How do you prevent too many API calls?

Approach: Implement debouncing — wait for the user to stop typing (e.g., 300ms pause) before firing the API call. Use `debounce()` function wrapping the fetch call. Also consider cancelling the previous request using `AbortController` when a new one starts.

3. A form submission is causing the page to refresh. How do you fix this?

Approach: Call `event.preventDefault()` inside the submit event handler to prevent the browser's default form submission behavior. Then handle the form data manually via JavaScript/fetch.

4. You have a REST API endpoint that is throwing a 401 error. How do you handle this in your frontend?

Approach: Intercept API responses globally (e.g., in an Axios interceptor or fetch wrapper). On 401, attempt to refresh the auth token. If refresh fails, redirect the user to the login page and clear stored tokens.

5. A list of 10,000 items needs to be displayed in the UI. How do you handle this efficiently?

Approach: Use virtual scrolling / windowing (libraries like `react-window` or `react-virtual`). Only render items visible in the viewport. Alternatively, implement pagination or infinite scroll with server-side data fetching.

6. How would you implement a 'dark mode' toggle for a website?

Approach: Use CSS custom properties (variables) for colors and toggle a class (e.g., 'dark') on the root element. Store preference in `localStorage`. Detect system preference with `window.matchMedia('(prefers-color-scheme: dark)')` on first load.

7. An API call inside a React `useEffect` is causing an infinite re-render loop. What is the issue and how do you fix it?

Approach: The dependency array of `useEffect` is either missing or incorrectly includes state that the effect modifies. Fix: ensure the dependency array only contains values the effect actually depends on, and avoid updating state that triggers the effect again without a condition.

8. You need to share state between two sibling React components. How would you approach this?

Approach: Lift state up to the nearest common parent and pass it via props. For complex or deeply nested apps, use React Context API or a state management library like `Redux` or `Zustand` to avoid prop drilling.

9. A user reports that clicking a button works on their desktop browser but not on their mobile device. How do you investigate?

Approach: Check for touch vs click event differences. Test using Chrome DevTools device emulation. Consider using pointer events or ensuring the click target is large enough. Check for hover-dependent CSS that may not work on touch devices. Test on real device via remote debugging.

10. You've deployed a JavaScript application, but users are seeing an old cached version. How do you force browsers to load the updated files?

Approach: Use cache-busting techniques: append a hash or version number to filenames (e.g., `app.abc123.js`). Configure server `Cache-Control` headers properly. Most bundlers (`Webpack`, `Vite`) do content hashing automatically. Set appropriate `max-age` and `ETag` headers on the server.

Section 5 · HR / Behavioral Questions

10 Questions on Communication, Teamwork & Attitude

1. Tell me about yourself.

Tip: Structure using the Present-Past-Future formula: current skills/projects → relevant background → why you're excited about this role. Keep it under 2 minutes. Focus on technical achievements.

2. Why do you want to join Celestial Systems?

Tip: Research the company beforehand. Mention specific alignment with their product/tech stack. Express interest in the learning opportunity during the 3-month training program.

3. Describe a time you faced a technical challenge and how you solved it.

Tip: Use the STAR method: Situation, Task, Action, Result. Be specific — mention the technology, the bug/problem, and the concrete steps you took. Quantify the outcome if possible.

4. How do you handle working under tight deadlines?

Tip: Talk about prioritization, breaking tasks into smaller milestones, communicating proactively with the team about blockers, and maintaining code quality even under pressure.

5. Have you ever disagreed with a teammate or senior? How did you handle it?

Tip: Show maturity: listen first, then present your perspective with data/reasoning. Emphasize collaboration and reaching a consensus. Avoid talking negatively about past colleagues.

6. Where do you see yourself in 2–3 years?

Tip: Align your answer with growth: becoming a full-stack developer, taking on more complex architecture decisions, potentially leading a small team, and contributing to open source or AI-integrated products.

7. How do you keep up with the latest JavaScript trends and technologies?

Tip: Mention: following blogs (MDN, Dev.to, JavaScript Weekly), building side projects, watching conference talks (JSConf), contributing to GitHub, and practicing on platforms like LeetCode or Frontend Mentor.

8. How do you approach learning a new technology or framework quickly?

Tip: Read official docs first, then build a small project with it. Use YouTube/courses for fundamentals, then dive into real codebases on GitHub. Ask questions and share learnings with peers.

9. What is your greatest strength and weakness as a developer?

Tip: Strength: be genuine (e.g., problem-solving, attention to detail). Weakness: pick something real but non-critical (e.g., 'I used to overthink edge cases' — then show how you've improved).

10. Do you have any questions for us?

Tip: Always ask questions! Good ones: 'What does the day-to-day look like for a JS developer here?', 'What is the tech stack used in production?', 'How is performance measured during the training period?'

Pro Tip: Bring your personal laptop charged and ready · Arrive 15 minutes early · Carry your resume (3 copies) · Dress smartly · Practice coding on paper/whiteboard beforehand.